

Contents

SECTION 15: EXTERNAL CREDENTIALS REGISTRY (v3.2.0)	1
	1
15.1 Credentials Registry Overview	1
15.2 Credentials Database Schema	1
15.3 Credentials Manager Service	2
	5

SECTION 15: EXTERNAL CREDENTIALS REGISTRY (v3.2.0)

15.1 Credentials Registry Overview

Secure storage and management of external API credentials with encryption, rotation, and access auditing.

15.2 Credentials Database Schema

```
-- migrations/025_credentials_registry.sql
```

```
CREATE TABLE credential_vaults (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    tenant_id UUID NOT NULL REFERENCES tenants(id),  
    vault_name VARCHAR(100) NOT NULL,  
    description TEXT,  
    encryption_key_arn VARCHAR(500) NOT NULL,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE(tenant_id, vault_name)  
);  
  
CREATE TABLE stored_credentials (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
    vault_id UUID NOT NULL REFERENCES credential_vaults(id) ON DELETE CASCADE,  
    credential_name VARCHAR(100) NOT NULL,  
    credential_type VARCHAR(50) NOT NULL,  
    encrypted_value BYTEA NOT NULL,  
    metadata JSONB DEFAULT '{}',  
    expires_at TIMESTAMPTZ,  
    last_rotated TIMESTAMPTZ,  
    rotation_schedule VARCHAR(50),  
    is_active BOOLEAN DEFAULT true,  
    created_by UUID REFERENCES users(id),  
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

```

CREATE TABLE credential_access_log (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    credential_id UUID NOT NULL REFERENCES stored_credentials(id),
    accessed_by UUID REFERENCES users(id),
    access_type VARCHAR(50) NOT NULL,
    access_reason TEXT,
    source_ip VARCHAR(45),
    user_agent TEXT,
    success BOOLEAN NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_credentials_vault ON stored_credentials(vault_id);
CREATE INDEX idx_credential_access_log ON credential_access_log(credential_id, created_at DESC);

ALTER TABLE credential_vaults ENABLE ROW LEVEL SECURITY;
ALTER TABLE stored_credentials ENABLE ROW LEVEL SECURITY;
ALTER TABLE credential_access_log ENABLE ROW LEVEL SECURITY;

CREATE POLICY vault_isolation ON credential_vaults USING (tenant_id = current_setting('app.current_tenant_id'));
CREATE POLICY credentials_isolation ON stored_credentials USING (
    vault_id IN (SELECT id FROM credential_vaults WHERE tenant_id = current_setting('app.current_tenant_id'))
);
CREATE POLICY access_log_isolation ON credential_access_log USING (
    credential_id IN (
        SELECT sc.id FROM stored_credentials sc
        JOIN credential_vaults cv ON sc.vault_id = cv.id
        WHERE cv.tenant_id = current_setting('app.current_tenant_id')::UUID
    )
);

```

15.3 Credentials Manager Service

```

// packages/core/src/services/credentials-manager.ts

import { Pool } from 'pg';
import { KMSSClient, EncryptCommand, DecryptCommand, GenerateDataKeyCommand } from '@aws-sdk/client-kms';
import { SecretsManagerClient, GetSecretValueCommand, UpdateSecretCommand } from '@aws-sdk/client-secretsmanager';

export class CredentialsManager {
    private pool: Pool;
    private kms: KMSSClient;
    private secrets: SecretsManagerClient;

    constructor(pool: Pool) {
        this.pool = pool;
        this.kms = new KMSSClient({});
    }
}

```

```

        this.secrets = new SecretsManagerClient({});
    }

    async createVault(
        tenantId: string,
        vaultName: string,
        description?: string
    ): Promise<string> {
        const keyArn = await this.createKmsKey(tenantId, vaultName);

        const result = await this.pool.query(`
            INSERT INTO credential_vaults (tenant_id, vault_name, description, encryption_key_arn)
            VALUES ($1, $2, $3, $4)
            RETURNING id
        `, [tenantId, vaultName, description, keyArn]);

        return result.rows[0].id;
    }

    async storeCredential(
        vaultId: string,
        name: string,
        type: string,
        value: string,
        metadata?: Record<string, any>,
        createdBy?: string
    ): Promise<string> {
        const vault = await this.getVault(vaultId);
        const encrypted = await this.encrypt(value, vault.encryption_key_arn);

        const result = await this.pool.query(`
            INSERT INTO stored_credentials (vault_id, credential_name, credential_type, encryption_key_arn, encrypted_value)
            VALUES ($1, $2, $3, $4, $5)
            RETURNING id
        `, [vaultId, name, type, encrypted, JSON.stringify(metadata || {}), createdBy]);

        return result.rows[0].id;
    }

    async getCredential(
        credentialId: string,
        accessedBy: string,
        accessReason: string,
        sourceIp?: string
    ): Promise<string> {
        const cred = await this.pool.query(`
            SELECT sc.*, cv.encryption_key_arn
            FROM stored_credentials sc
            JOIN credential_vaults cv ON cv.tenant_id = sc.tenant_id AND cv.vault_name = sc.vault_name
        `);
    }

```

```

        JOIN credential_vaults cv ON sc.vault_id = cv.id
        WHERE sc.id = $1 AND sc.is_active = true
    `, [credentialId]);

    if (cred.rows.length === 0) {
        await this.logAccess(credentialId, accessedBy, 'read', accessReason, sourceIp, false);
        throw new Error('Credential not found or inactive');
    }

    const { encrypted_value, encryption_key_arn } = cred.rows[0];
    const decrypted = await this.decrypt(encrypted_value, encryption_key_arn);

    await this.logAccess(credentialId, accessedBy, 'read', accessReason, sourceIp, true);

    return decrypted;
}

async rotateCredential(
    credentialId: string,
    newValue: string,
    rotatedBy: string
): Promise<void> {
    const cred = await this.pool.query(`
        SELECT sc.*, cv.encryption_key_arn
        FROM stored_credentials sc
        JOIN credential_vaults cv ON sc.vault_id = cv.id
        WHERE sc.id = $1
    `, [credentialId]);

    if (cred.rows.length === 0) throw new Error('Credential not found');

    const encrypted = await this.encrypt(newValue, cred.rows[0].encryption_key_arn);

    await this.pool.query(`
        UPDATE stored_credentials
        SET encrypted_value = $2, last_rotated = NOW(), updated_at = NOW()
        WHERE id = $1
    `, [credentialId, encrypted]);

    await this.logAccess(credentialId, rotatedBy, 'rotate', 'Credential rotation', null, true);
}

private async encrypt(plaintext: string, keyArn: string): Promise<Buffer> {
    const response = await this.kms.send(new EncryptCommand({
        KeyId: keyArn,
        Plaintext: Buffer.from(plaintext)
    }));
    return Buffer.from(response.CiphertextBlob!);
}

```

```

    }

    private async decrypt(ciphertext: Buffer, keyArn: string): Promise<string> {
        const response = await this.kms.send(new DecryptCommand({
            KeyId: keyArn,
            CiphertextBlob: ciphertext
        }));
        return Buffer.from(response.Plaintext!).toString();
    }

    private async createKmsKey(tenantId: string, vaultName: string): Promise<string> {
        // In production, create a new KMS key
        return `arn:aws:kms:us-east-1:${process.env.AWS_ACCOUNT_ID}:key/${tenantId}-${vaultName}`;
    }

    private async getVault(vaultId: string) {
        const result = await this.pool.query(`SELECT * FROM credential_vaults WHERE id = $1`,
        if (result.rows.length === 0) throw new Error('Vault not found');
        return result.rows[0];
    }

    private async logAccess(
        credentialId: string,
        accessedBy: string,
        accessType: string,
        reason: string,
        sourceIp: string | null,
        success: boolean
    ): Promise<void> {
        await this.pool.query(`
            INSERT INTO credential_access_log (credential_id, accessed_by, access_type, access_reason, source_ip, success)
            VALUES ($1, $2, $3, $4, $5, $6)
        `, [credentialId, accessedBy, accessType, reason, sourceIp, success]);
    }
}

```